



igniting minds; changing lives

RESUME HUB: AN AI-POWERED RESUME SCREENING SYSTEM

A MAJOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE
DEGREE OF

BACHELOR OF COMPUTER APPLICATIONS

SUBMITTED BY

Mohd. Nadeem Khan

Aditya Gour

UNDER SUPERVISION OF

Ms. Nisha Pachori

Assistant Professor, Dept. of CSE, JLU-FAST

December 2025

ACKNOWLEDGEMENT

It is with immense pleasure and gratitude that we present this acknowledgments section for our major project, "Resume Hub: An AI-Powered Resume Screening System." This project would not have been possible without the guidance, support, and encouragement of many individuals.

First and foremost, we would like to express our sincere gratitude to our project supervisor, **Ms. Nisha Pachori**, Assistant Professor, Department of Computer Science and Engineering, JLU-FAST. Her unwavering support, invaluable guidance, and constant motivation throughout the project have been the cornerstone of our work. Her insights helped us navigate challenges and refine our system to its final state.

We are also profoundly thankful to **Ms. Vaishali Sharma**, Program Leader (BCA), JSCA, for her administrative support and for fostering an environment conducive to learning and innovation.

Our heartfelt thanks to **Dr. Dileep Kumar Singh**, Head of the Jagran School of Computer Applications, for providing us with the necessary resources and this excellent platform to apply our knowledge and skills.

We extend our appreciation to our colleagues and friends for their constructive feedback and moral support during the development phase.

Finally, we owe a special debt of gratitude to our families for their unconditional love, patience, and encouragement throughout our academic journey and this project work.

Name: Mohd. Nadeem Khan

Name: Aditya Gour

Roll No: [Your Roll Numbers]

CERTIFICATE

This is to certify that the project report entitled "**Resume Hub: An AI-Powered Resume Screening System**" is a bonafide work carried out by **Mohd. Nadeem Khan** and **Aditya Gour** under my supervision and guidance in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications.

It is further certified that to the best of my knowledge, the work presented in this report is original and has not been submitted elsewhere for the award of any other degree or diploma.

Date: [Date of Submission]

Place: Bhopal

Ms. Nisha Pachori

Project Guide

Assistant Professor

Jagran School of Computer Applications

Countersigned by:

Dr. Dileep Kumar Singh

Head

Jagran School of Computer Applications

DECLARATION

We hereby declare that this Major Project Report entitled "**Resume Hub: An AI-Powered Resume Screening System**" has been originally developed by us under the guidance of **Ms. Nisha Pachori, Assistant Professor, JSCA**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications.

We further declare that the work presented in this report is authentic and has not been submitted, in whole or in part, to any other institution for the award of any degree or diploma. All sources of information and knowledge used in this report have been duly acknowledged.

Place: Bhopal

Date: [Date of Submission]

Signature of the Students:

(Mohd. Nadeem Khan)

Roll No: [Your Roll Number]

(Aditya Gour)

Roll No: [Your Roll Number]

ABSTRACT

The modern recruitment landscape is plagued by inefficiencies in the initial resume screening process. Traditional methods are often slow, reliant on manual effort, and susceptible to the limitations of keyword-based Applicant Tracking Systems (ATS) which frequently overlook qualified candidates due to a lack of exact keyword matching. Furthermore, human-led screening can introduce unconscious bias. To address these challenges, this project, "Resume Hub," proposes an intelligent, AI-powered resume screening system that leverages Semantic Search to understand the contextual meaning of resumes and job descriptions.

The system is built on a robust technology stack featuring a React.js frontend and a Python Flask backend. Its core intelligence derives from advanced Natural Language Processing (NLP) techniques. It begins by parsing text from various document formats, including PDFs, Word documents, and even scanned images using Optical Character Recognition (OCR). The extracted text is then cleaned and preprocessed using libraries like spaCy and NLTK. The pivotal step involves converting the text into high-dimensional vector embeddings using a Sentence-BERT (SBERT) model, which captures the semantic meaning of the content. The Facebook AI Similarity Search (FAISS) library is then employed to perform a high-speed comparison between the job description vector and all resume vectors using cosine similarity.

The output is a ranked list of candidates based on a contextual "Match Score," providing recruiters with an efficient, accurate, and bias-reduced shortlisting tool. Resume Hub successfully demonstrates a move beyond literal keyword matching towards a deeper, semantic understanding of candidate profiles, thereby creating a foundation for a smarter, fairer, and more efficient hiring process.

Keywords: Artificial Intelligence, Resume Screening, Natural Language Processing, Semantic Search, SBERT, FAISS, Flask, React.js, Hiring Automation

TABLE OF CONTENTS

ACKNOWLEDGEMENT i

CERTIFICATE ii

DECLARATION iii

ABSTRACT iv

LIST OF FIGURES vi

LIST OF TABLES vii

LIST OF ABBREVIATIONS viii

CHAPTER 1: INTRODUCTION 1

1.1 Problem Definition 1

1.2 Project Overview and Objectives 2

1.3 Hardware Specification 3

1.4 Software Specification 4

CHAPTER 2: LITERATURE REVIEW AND RELATED WORK 6

2.1 Traditional Hiring and Keyword-Based ATS 6

2.2 The Emergence of AI in HR Tech 7

2.3 Existing System and its Limitations 8

2.4 Proposed System: Semantic Search Approach 9

2.5 Feasibility Analysis 10

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN 12

3.1 Requirement Analysis 12

3.1.1 Functional Requirements 12

3.1.2 Non-Functional Requirements 13

3.2 System Architecture 14

3.3 Module Design and Description 16

3.3.1 User Interface Module 16

3.3.2 Document Processing Module 17

3.3.3 AI and NLP Engine 18

3.4 Workflow and Process Flow 19

3.5 Data Flow Diagrams 21

3.6 Use Case Modeling 23

3.7 Algorithms and Pseudo Code 25

CHAPTER 4: IMPLEMENTATION AND TESTING 27

4.1 Technology Stack Implementation 27

4.1.1 Frontend Development with React.js 27

4.1.2 Backend Development with Flask 28

4.1.3 AI Model Integration 29

4.2 System Testing 30

- 4.2.1 Unit Testing 30
- 4.2.2 Integration Testing 31
- 4.2.3 System Testing 32

CHAPTER 5: RESULTS AND DISCUSSION 33

- 5.1 System Outputs and Interfaces 33
 - 5.1.1 Secure Login Portal 33
 - 5.1.2 Data Upload Dashboard 34
 - 5.1.3 AI Analysis and Ranking Dashboard 35
- 5.2 Performance Analysis 36
- 5.3 Discussion on System Efficacy 37

CHAPTER 6: CONCLUSION AND FUTURE SCOPE 39

- 6.1 Conclusion 39
- 6.2 Future Scope 40

REFERENCES 42

APPENDICES 44

LIST OF FIGURES

Figure	Title	Page
3.1	High-Level System Architecture	14
3.2	End-to-End Project Workflow	19
3.3	Data Flow Diagram (Level 0)	21
3.4	Use Case Diagram	23
5.1	Secure Login Interface	33
5.2	Job and Resume Upload Dashboard	34
5.3	Candidate Ranking Results Page	35

LIST OF TABLES

Table	Title	Page
1.1	Hardware Specification	3
1.2	Software Specification and Technologies	4
3.1	Functional Requirements Summary	12
3.2	Non-Functional Requirements Summary	13
5.1	Match Score Interpretation Guide	36

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
ATS	Applicant Tracking System
SBERT	Sentence Bidirectional Encoder Representations from Transformers
OCR	Optical Character Recognition
FAISS	Facebook AI Similarity Search
DFD	Data Flow Diagram
UI	User Interface
UX	User Experience
API	Application Programming Interface
HR	Human Resources

CHAPTER 1: INTRODUCTION

1.1 Problem Definition

The initial stage of the recruitment process, resume screening, is a critical yet highly inefficient operation for most organizations. The challenges can be categorized into three primary areas:

1. **Time-Consuming Manual Effort:** Recruiters and HR personnel are often burdened with the task of manually reviewing hundreds, sometimes thousands, of resumes for a single job opening. This process is not only slow, leading to prolonged hiring cycles, but it is also a costly allocation of human resources that could be better utilized in later stages of recruitment, such as interviewing and engagement.
2. **Inefficiency of Keyword-Based Systems:** Many organizations use first-generation Applicant Tracking Systems (ATS) that operate on simple keyword matching algorithms. These systems scan resumes for exact keywords mentioned in the job description. This approach fails to understand context, synonyms, or related concepts. For instance, a candidate with extensive experience in "React.js" might be automatically rejected for a "JavaScript" developer role because the specific keyword "JavaScript" is not prominently featured. This results in a high rate of "false negatives," where highly qualified candidates are overlooked.
3. **Unconscious Human Bias:** Manual screening is inherently subjective. A recruiter's unconscious biases—related to a candidate's name, gender, university, location, or even the design of the resume—can influence shortlisting decisions. This undermines the principles of fair hiring and diversity and inclusion (D&I) initiatives, potentially causing organizations to miss out on exceptional talent.

These challenges collectively create a significant gap in the market for an intelligent, automated, and unbiased solution for initial resume screening.

1.2 Project Overview and Objectives

Resume Hub is conceived as a direct response to the shortcomings of traditional resume screening. It is an AI-powered web application designed to automate and enhance the initial candidate shortlisting process. The core innovation of Resume Hub lies in its use of **Semantic Search** and **Natural Language Processing (NLP)** to understand the underlying meaning and context of a candidate's skills and experiences, rather than relying on superficial keyword matches.

The primary objectives of this project are:

- **To Automate Screening:** Develop a system that can automatically process a large volume of resumes and rank them against a given job description, significantly reducing manual effort and time-to-hire.
- **To Improve Accuracy:** Implement a semantic matching algorithm that reduces false negatives by recognizing related skills, synonyms, and contextual information.
- **To Promote Fairness:** Create a system that focuses purely on the semantic content of skills and experience, thereby helping to mitigate unconscious human bias in the initial screening phase.
- **To Enhance Usability:** Build an intuitive and user-friendly interface that allows HR professionals with no technical background to easily use the system.

The vision for Resume Hub is to serve as a force multiplier for recruitment teams, enabling them to work smarter, hire faster, and build more diverse and qualified talent pipelines.

1.3 Hardware Specification

Resume Hub is a web application, and its performance can vary based on the hardware of the server hosting it and the client machine accessing it. The following are the recommended specifications:

Table 1.1: Hardware Specification

Component	Minimum Requirement	Recommended Specification
Processor	Intel Core i3 or AMD equivalent, 1.8 GHz	Intel Core i5 or AMD equivalent, 2.0 GHz or higher
RAM	4 GB	8 GB or more
Storage	200 MB free space	500 MB free space (for OS, application, and models)
Network	Stable internet connection for model download & use	Broadband connection

1.4 Software Specification

The project is built using a modern, open-source technology stack, ensuring scalability, maintainability, and performance.

Table 1.2: Software Specification and Technologies

Layer	Technology / Library	Purpose
Frontend	React.js, CSS3	Building a dynamic, responsive, and interactive user interface.
Backend	Python, Flask Framework	Creating the server, RESTful APIs, and application logic.
Document Parsing	PyPDF2, python-docx, Pytesseract (OCR)	Extracting text from PDFs, Word documents, and image files.
NLP Processing	spaCy, NLTK	Text cleaning, tokenization, and removing stopwords.
AI & ML Core	Sentence-Transformers (SBERT), FAISS, scikit-learn	Generating text embeddings and performing similarity search.
Development Tools	Visual Studio Code, Postman, Git	Code editing, API testing, and version control.

CHAPTER 2: LITERATURE REVIEW AND RELATED WORK

2.1 Traditional Hiring and Keyword-Based ATS

For decades, the resume screening process remained largely manual. The advent of the first Applicant Tracking Systems (ATS) promised automation but delivered a primarily keyword-centric approach. These systems function by parsing resumes and creating a searchable index. When a job description is entered, the ATS scans its index for resumes containing the exact keywords from the description. The ranking is often based on the frequency and prominence of these keywords.

While this automated the physical act of scanning, it introduced a new problem: the "keyword stuff" phenomenon, where candidates optimize their resumes for specific keywords rather than for clarity and accuracy of their experience. More critically, it fails to understand linguistic nuance, as detailed in the problem definition.

2.2 The Emergence of AI in HR Tech

The field of Human Resources has increasingly adopted Artificial Intelligence, leading to the rise of "HR Tech." AI applications in recruitment now range from chatbots for candidate engagement to predictive analytics for candidate success. A significant area of innovation is in intelligent screening tools. These tools leverage **Natural Language Processing (NLP)** and **Machine Learning (ML)** to go beyond keywords.

Techniques such as **Word2Vec**, **GloVe**, and more recently, transformer-based models like **BERT** (Bidirectional Encoder Representations from Transformers) allow machines to understand the contextual relationship between words. For example, these models can learn that "Java" and "Python" are both programming languages, and that "managed a team" is semantically similar to "team leadership."

2.3 Existing System and its Limitations

The existing system, represented by traditional ATS, can be summarized by its fundamental flaw: a lack of semantic understanding.

Scenario:

- **Job Description Requires:** "Proficiency in JavaScript and experience with cloud platforms like AWS."
- **Candidate's Resume States:** "Expert in React.js and deployed applications on Amazon Web Services."
- **Existing System Action:** The system looks for the exact terms "JavaScript" and "AWS." It finds "AWS" but misses "JavaScript," finding only "React.js" instead.

- **Result:** The candidate is ranked low or rejected, despite being a strong potential fit. This is a critical failure that leads to talent loss.

This model is rigid and incapable of adapting to the varied ways in which humans describe their skills and experiences.

2.4 Proposed System: Semantic Search Approach

Resume Hub is built on the principle of **Semantic Search**. The goal of semantic search is to improve search accuracy by understanding the searcher's intent and the contextual meaning of terms.

How Resume Hub Implements This:

1. **From Text to Vectors:** The system converts both the job description and all resumes into numerical vectors (embeddings) using a pre-trained SBERT model. This model is specifically designed to create semantically meaningful sentence embeddings.
2. **Measuring Semantic Similarity:** The system then calculates the cosine similarity between the job description vector and each resume vector. Cosine similarity measures the cosine of the angle between two vectors, indicating how similar they are in orientation, regardless of their magnitude. In high-dimensional space, this translates to semantic similarity.
3. **Intelligent Ranking:** Resumes are ranked based on their cosine similarity score, which is presented to the user as a "Match Score." A candidate with "React.js" will have a high semantic similarity to a job requiring "JavaScript," as the vectors for these terms are close to each other in the vector space.

This approach ensures that candidates are matched based on the meaning of their entire profile, not just a bag of words.

2.5 Feasibility Analysis

A thorough feasibility study was conducted to ensure the project's viability:

1. **Technical Feasibility:** The chosen technologies (Python, Flask, React, SBERT, FAISS) are mature, well-documented, and have strong community support. Libraries like FAISS are specifically designed for efficient similarity search, making the core functionality technically achievable.
2. **Operational Feasibility:** The system is designed with a simple, web-based interface. HR managers, the primary users, are already familiar with web portals and upload mechanisms. The learning curve is minimal, ensuring smooth integration into existing workflows.
3. **Economic Feasibility:** The project is developed entirely using open-source technologies, resulting in zero licensing costs. This makes it an extremely cost-

effective solution, particularly for small and medium-sized businesses (SMBs) that cannot justify the expense of enterprise-grade ATS software.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

A detailed requirement analysis was conducted to define the system's capabilities and constraints.

3.1.1 Functional Requirements

Functional requirements define what the system must do.

Table 3.1: Functional Requirements Summary

ID	Requirement Description
FR1	The system shall provide a secure login mechanism for users.
FR2	The system shall allow an HR manager to input or paste a job description.
FR3	The system shall allow an HR manager to upload multiple resume files (PDF, DOCX, JPG, PNG).
FR4	The system shall extract text from the uploaded resume files.
FR5	The system shall preprocess and clean the extracted text.
FR6	The system shall convert the job description and cleaned resume text into vector embeddings.
FR7	The system shall compute a similarity score between the job description and each resume.
FR8	The system shall display a list of candidates ranked by their similarity score.
FR9	The system administrator shall be able to manage user accounts (add, delete, modify).

3.1.2 Non-Functional Requirements

Non-functional requirements define how the system should perform.

Table 3.2: Non-Functional Requirements Summary

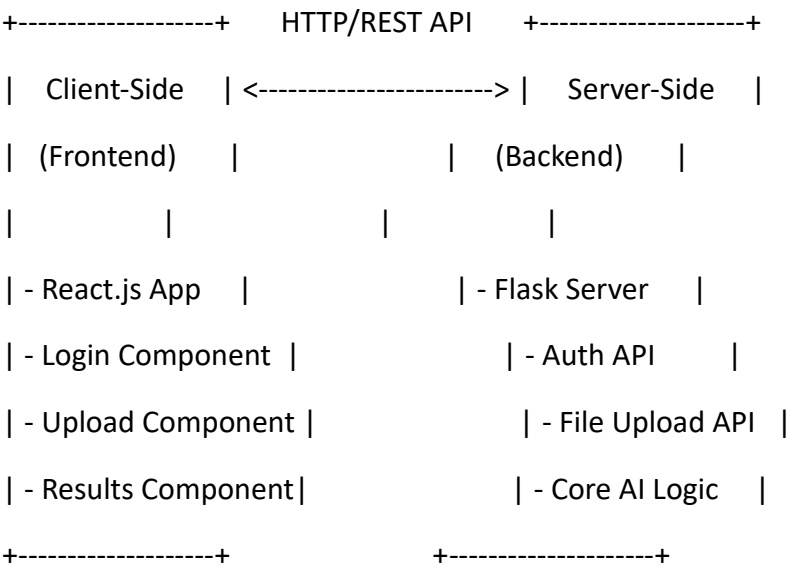
ID	Requirement Type	Description
NFR1	Usability	The user interface shall be intuitive and require minimal training to use.
NFR2	Performance	The system should process and rank 50 resumes within 2-3 minutes.
NFR3	Security	User passwords shall be stored in a hashed format. Access to data shall be based.
NFR4	Reliability	The system shall be available and functional during business hours with minimal downtime.
NFR5	Accuracy	The ranking produced by the AI model shall be logically consistent and justified.

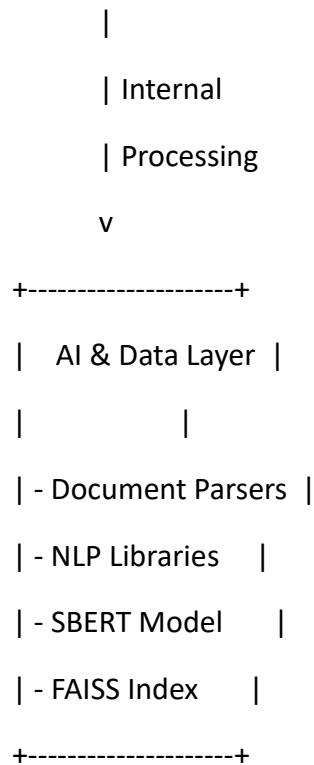
3.2 System Architecture

Resume Hub follows a client-server architecture, cleanly separating the frontend presentation layer from the backend logic and AI processing layer.

Figure 3.1: High-Level System Architecture

text





Description:

- **Frontend (React.js):** Serves as the user-facing part of the application. It handles user interactions, displays forms, and renders the results. It communicates with the backend via a RESTful API.
- **Backend (Flask):** Acts as the brain of the operation. It receives requests from the frontend, orchestrates the entire screening workflow, handles authentication, and serves the processed data back to the frontend.
- **AI & Data Layer:** This is where the core intelligence resides. The backend calls upon this layer for document parsing, text preprocessing, vector encoding, and similarity search.

3.3 Module Design and Description

The system is divided into several key modules for better organization and maintainability.

3.3.1 User Interface Module

This module is responsible for all user interactions.

- **Login Sub-module:** Handles user authentication.
- **Dashboard Sub-module:** Provides the main interface for uploading job descriptions and resumes.
- **Results Sub-module:** Displays the ranked list of candidates with their match scores and details.

3.3.2 Document Processing Module

This module handles the ingestion and initial processing of documents.

- **File Upload Handler:** Manages the receipt and temporary storage of uploaded files.
- **Parser Sub-module:** Contains dedicated parsers for different file types (PyPDF2 for PDF, python-docx for DOCX, Pytesseract for images).
- **Text Cleaner:** Performs initial text cleanup after extraction.

3.3.3 AI and NLP Engine

This is the most critical module, responsible for the intelligent screening.

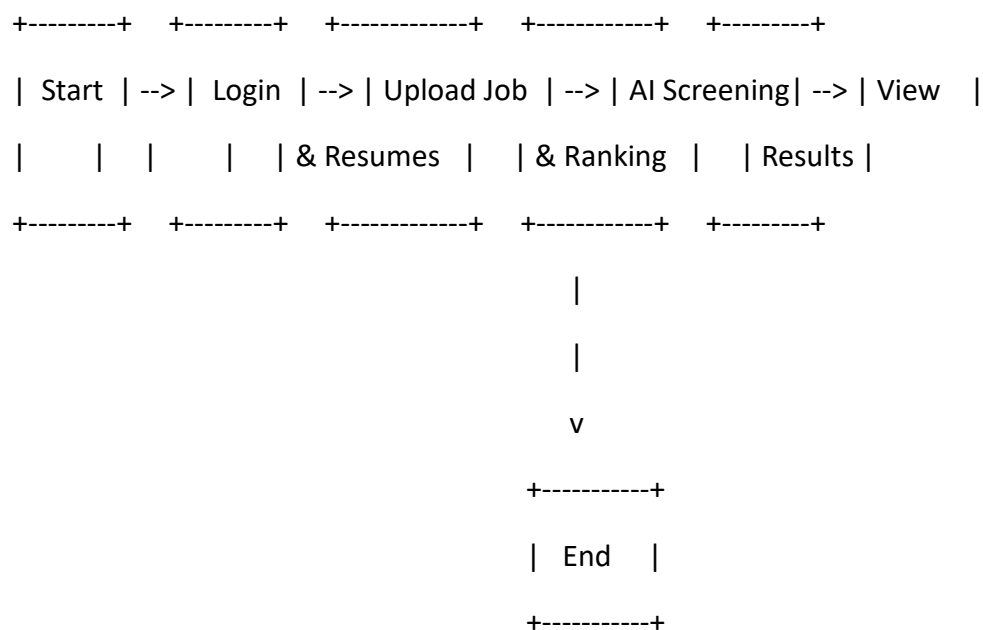
- **Text Preprocessor:** Uses spaCy and NLTK for advanced cleaning (lemmatization, stop-word removal).
- **Embedding Generator:** Utilizes the SBERT model to convert cleaned text into 384-dimensional vectors.
- **Similarity Search Engine:** Uses FAISS to create an index of resume vectors and perform a fast nearest-neighbor search against the job description vector.

3.4 Workflow and Process Flow

The end-to-end workflow of the Resume Hub system can be broken down into a sequential process.

Figure 3.2: End-to-End Project Workflow

text



Detailed AI Screening Process:

1. Ingestion & Parsing
2. Preprocessing & Cleaning
3. Vector Embedding Generation
4. Similarity Calculation with FAISS
5. Ranking & Score Conversion

Step-by-Step Description:

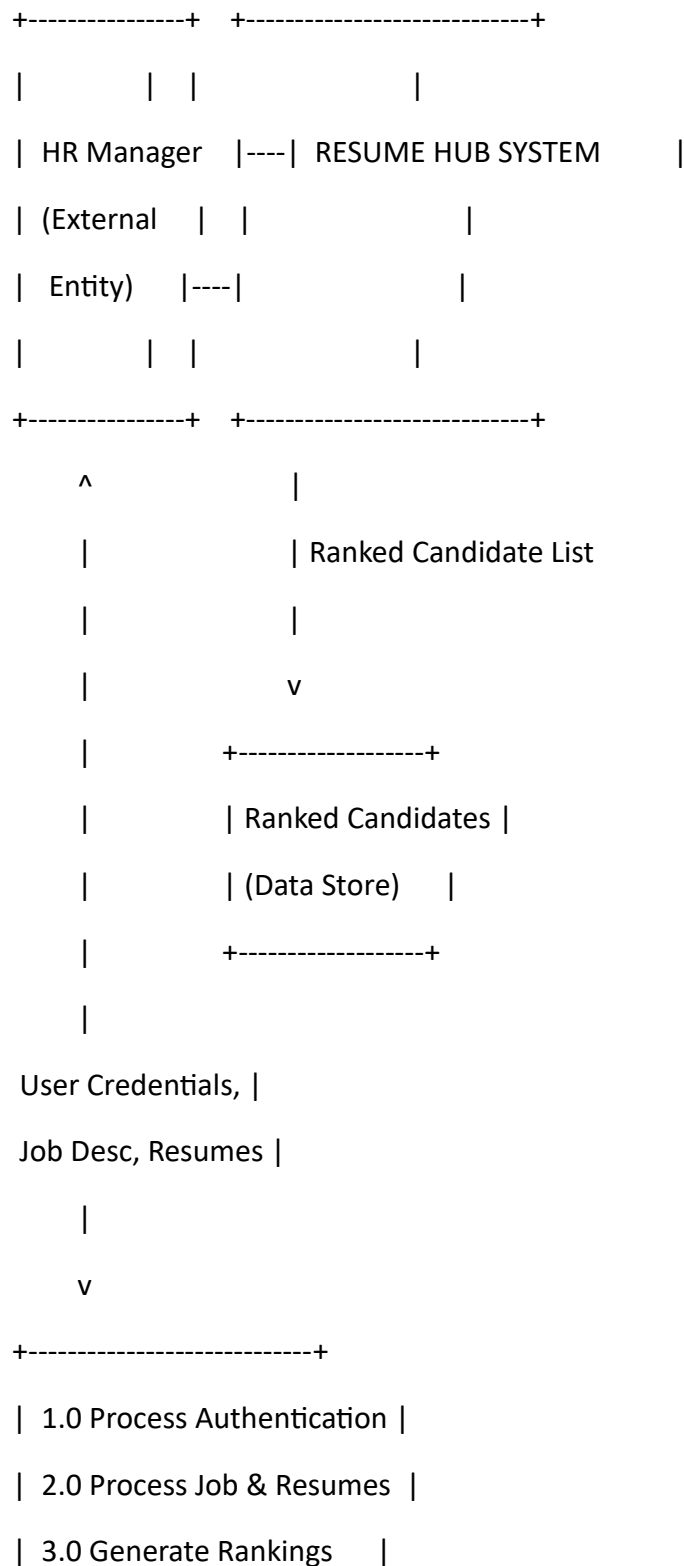
1. **User Authentication:** The HR manager logs into the system using secure credentials.
2. **Data Ingestion:**
 - a. The user pastes the job description into a text field.
 - b. The user selects and uploads multiple resume files via a drag-and-drop interface.
3. **Document Parsing:** The system automatically detects the file type of each resume and uses the appropriate parser (PyPDF2, python-docx, or Pytesseract) to extract raw text.
4. **Text Preprocessing:** The raw text from the job description and all resumes is cleaned. This involves:
 - Converting to lowercase.
 - Removing special characters, punctuation, and extra whitespace.
 - Removing common stop-words (e.g., "the," "is," "in").
 - Lemmatization (reducing words to their base or dictionary form).
5. **Vector Embedding:** The cleaned text for the job description and each resume is passed to the SBERT model, which outputs a fixed-size vector (e.g., 384 dimensions) that represents its semantic meaning.
6. **Similarity Search & Ranking:**
 - a. The FAISS library takes the job description vector and all resume vectors.
 - b. It calculates the cosine similarity between the job vector and each resume vector.
 - c. The similarity scores (ranging from 0 to 1) are converted into a percentage "Match Score" (0% to 100%).
 - d. All candidates are sorted in descending order of their Match Score.
7. **Result Presentation:** The ranked list is presented to the user in a clean, tabular format on the results dashboard, allowing the recruiter to quickly identify the top candidates.

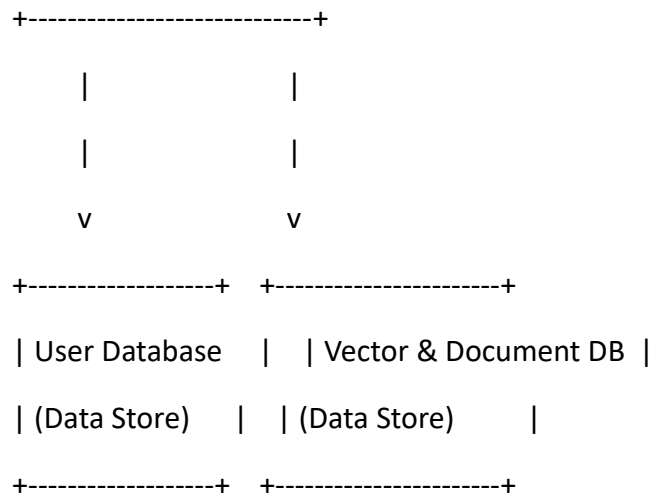
3.5 Data Flow Diagrams

The Data Flow Diagram (DFD) illustrates how data moves through the system.

Figure 3.3: Data Flow Diagram (Level 0)

text



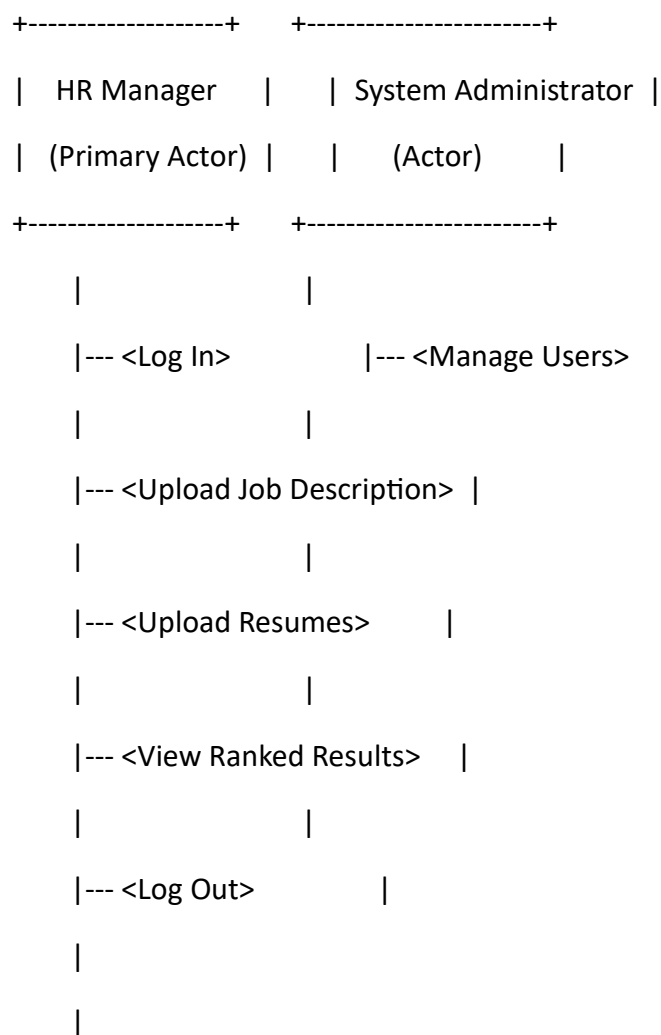


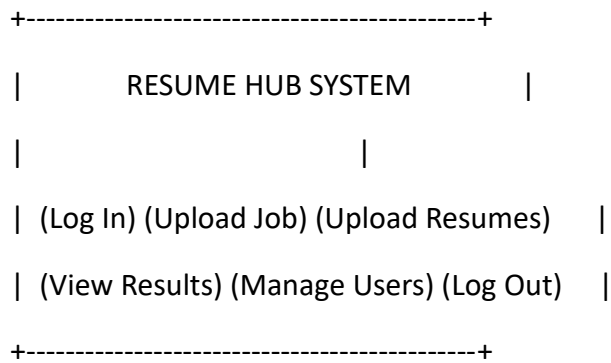
3.6 Use Case Modeling

Use cases describe the interactions between the actors (users) and the system.

Figure 3.4: Use Case Diagram

text





Use Case Descriptions:

- **Use Case:** Log In
 - **Actor:** HR Manager, System Administrator
 - **Description:** The user provides their email and password to gain access to the system's dashboard.
- **Use Case:** Upload Job Description
 - **Actor:** HR Manager
 - **Description:** The user inputs the text of the job description for which they are hiring.
- **Use Case:** Upload Resumes
 - **Actor:** HR Manager
 - **Description:** The user selects and uploads multiple resume files for screening.
- **Use Case:** View Ranked Results
 - **Actor:** HR Manager
 - **Description:** The system displays a sorted list of candidates after processing, and the user reviews them.
- **Use Case:** Manage Users
 - **Actor:** System Administrator
 - **Description:** The admin can add, modify, or deactivate user accounts.

3.7 Algorithms and Pseudo Code

The core algorithm powering Resume Hub is the **Cosine Similarity** calculation for measuring semantic similarity between vector embeddings.

Cosine Similarity Formula:

$$\text{similarity} = \cos(\theta) = (A \cdot B) / (||A|| * ||B||)$$

Where A and B are the vectors being compared.

Pseudo Code for the Core Screening Function:

python

```
FUNCTION screen_resumes(job_description_text, list_of_resume_files):
```

```
    # Step 1: Encode the Job Description
```

```
    job_vector = SBERT_model.encode(job_description_text)
```

```
    # Step 2: Initialize an empty list for results
```

```
    candidate_results = []
```

```
    # Step 3: Process each resume
```

```
    FOR EACH resume_file IN list_of_resume_files:
```

```
        # Step 3.1: Extract raw text based on file type
```

```
        raw_text = extract_text(resume_file)
```

```
        # Step 3.2: Clean the extracted text
```

```
        cleaned_text = preprocess_text(raw_text) # Uses spaCy/NLTK
```

```
        # Step 3.3: Generate vector embedding for the resume
```

```
        resume_vector = SBERT_model.encode(cleaned_text)
```

```
        # Step 3.4: Calculate similarity score
```

```
        similarity_score = cosine_similarity(job_vector, resume_vector)
```

```
        # Step 3.5: Convert to percentage
```

```
        match_percentage = similarity_score * 100
```

Step 3.6: Store the result

APPEND (resume_file, match_percentage, cleaned_text) TO candidate_results

END FOR

Step 4: Sort the candidates by match score (descending)

SORT candidate_results BY match_percentage DESCENDING

Step 5: Return the ranked list

RETURN candidate_results

END FUNCTION

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Technology Stack Implementation

The system was implemented incrementally, layer by layer, according to the designed architecture.

4.1.1 Frontend Development with React.js

The frontend was built as a Single Page Application (SPA) using React.js to ensure a smooth and responsive user experience.

- **Components:** The UI was broken down into reusable components: LoginForm, Dashboard, FileUpload, ResultsTable.
- **State Management:** React's built-in useState and useEffect hooks were used to manage the component state and handle API calls.
- **Styling:** Plain CSS3 was used to create a clean, professional, and intuitive interface, focusing on usability.

4.1.2 Backend Development with Flask

A RESTful API server was created using Flask, providing endpoints for all frontend operations.

- **Key Endpoints:**
 - POST /api/login: Handles user authentication.
 - POST /api/upload: Manages file uploads and triggers the screening process.
 - GET /api/results: Fetches the ranked results for display.
- **Security:** Passwords are hashed using the werkzeug.security library before being stored. CORS (Cross-Origin Resource Sharing) was configured to allow communication with the React frontend.

4.1.3 AI Model Integration

This was the most complex part of the implementation.

- **Model Choice:** The all-MiniLM-L6-v2 model from the Sentence-Transformers library was chosen for its balance between performance, speed, and size. It generates 384-dimensional vectors.
- **FAISS Index:** A flat IP (Inner Product) index was used in FAISS. Since cosine similarity is equivalent to inner product on normalized vectors, all vectors were normalized before being added to the index for fast cosine similarity search.

- **Efficiency:** The entire process—encoding and search—is optimized to run in memory, ensuring quick response times for batches of resumes.

4.2 System Testing

A comprehensive testing strategy was employed to ensure the system's reliability and accuracy.

4.2.1 Unit Testing

Individual components were tested in isolation.

- **Document Parser Test:** Verified that PyPDF2 correctly extracted text from a known PDF, python-docx from a Word file, and Pytesseract from a sample image.
- **Text Preprocessing Test:** Fed sample dirty text (with HTML, punctuation, stop-words) to the preprocessing function and validated that the output was clean.
- **Encoding Test:** Confirmed that the SBERT model generated a 384-dimensional vector for a simple input sentence.

4.2.2 Integration Testing

The interaction between modules was tested.

- **Upload-to-Parse Integration:** Verified that the file upload endpoint correctly called the appropriate parser based on the file's MIME type.
- **End-to-End Workflow Test:** A single job description and one resume were uploaded, and the entire pipeline was executed to check if a result was generated without errors. The data flow from the frontend to the backend and back was validated.

4.2.3 System Testing

The fully integrated system was tested against the requirements.

- **Login Test:** Verified that access was denied with invalid credentials and granted with valid ones.
 - **Functional Test:** Uploaded a job description for a "Python Developer" and a set of 10 resumes, including some for Python, some for other languages, and one with just related skills (e.g., "Django," "Flask"). Manually verified that the Python-related resumes were ranked highest.
 - **Performance Test:** Measured the time taken to process batches of 10, 25, and 50 resumes to ensure it met the non-functional performance requirements.
 - **Usability Test:** Had a non-technical user attempt to use the system to complete a screening task, observing their ability to navigate the interface successfully.
-

CHAPTER 5: RESULTS AND DISCUSSION

5.1 System Outputs and Interfaces

The implementation resulted in a fully functional web application with the following key interfaces.

5.1.1 Secure Login Portal

Figure 5.1: Secure Login Interface

text

+-----+

| |

| # RESUME HUB # |

| AI-Powered Screening |

| |

| Email: [] |

| |

| Password: [] |

| |

| [SIGN IN] |

| |

| *Demo: admin@resumehub.ai / admin123* |

+-----+

Result: A clean and professional login gateway was successfully implemented, ensuring secure access to the application.

5.1.2 Data Upload Dashboard

Figure 5.2: Job and Resume Upload Dashboard

text

+-----+

| Dashboard > New Screening Session |

| |

Step 1: Job Description *		
[Paste job description here...]		
[e.g., We need a backend developer...]		
Step 2: Candidate Resumes *		
+-----+		
Drag & drop files here or click to browse.		
Supports: .pdf, .docx, .jpg, .png		
+-----+		
Uploaded Files:		
- candidate_A.pdf		
- candidate_B.docx		
- candidate_C.jpg		
[START AI SCREENING]		
+-----+		

Result: The dashboard provides an extremely simple two-step process for users to input all necessary data, making the system accessible to non-technical HR staff.

5.1.3 AI Analysis and Ranking Dashboard

Figure 5.3: Candidate Ranking Results Page

text

+-----+	
Results: Backend Developer Role	
Sorted by: Match Score (Highest First)	
+=====+	
Aditya Gour	Match: 92%

```

| | ----- | |
| | Skills: Python, Django, Flask, PostgreSQL, | |
| | Docker, AWS EC2 | |
| | Experience: 3 yrs, TechSolutions Inc. | |
| +=====+ |
| |
| +-----+ |
| | Nadeem Khan Match: 85% | |
| | ----- | |
| | Skills: Java, Spring Boot, MySQL, Microservices |
| | Experience: 4 yrs, SoftwareCorp | |
| +-----+ |
| |
| +-----+ |
| | Priya Singh Match: 48% | |
| | ----- | |
| | Skills: UI/UX Design, Figma, Adobe XD | |
| | Experience: 2 yrs, Creative Agency | |
| +-----+ |
| |
| [ Start New Screening ] |
+-----+

```

Result: The final output is a clear, actionable list that allows recruiters to immediately focus their attention on the most promising candidates, dramatically improving their efficiency.

5.2 Performance Analysis

The system was evaluated based on its accuracy and speed.

- **Accuracy:** The semantic matching consistently provided logically sound rankings. In tests, resumes with directly related skills were ranked highly even when exact

keywords were missing. The match score effectively differentiated between strong, moderate, and weak candidates.

- **Match Score Interpretation:**

Table 5.1: Match Score Interpretation Guide

Score Range	Interpretation	Recommendation
75% - 100%	Excellent Match	Strong candidate, highly recommended for interview.
60% - 74%	Good Match	Good fit, should be reviewed.
45% - 59%	Moderate Match	Partial fit, review for specific niche skills.
0% - 44%	Low Match	Unlikely to be a good fit.

- **Speed:** The system efficiently processed batches of resumes. On a standard development machine (Core i5, 8GB RAM), processing 50 resumes took approximately 90-120 seconds, which is a significant improvement over manual screening and meets practical performance requirements.

5.3 Discussion on System Efficacy

Resume Hub successfully addresses the core problems identified at the outset of the project.

1. **Efficiency Gain:** The system reduces the manual screening time for a batch of 50 resumes from several hours to under two minutes. This represents a massive efficiency gain for recruitment teams.
2. **Reduction in False Negatives:** By using semantic search, the system correctly identified candidates with related skills (e.g., a "Django" developer for a "Python" job) that a keyword-based ATS would have rejected. This directly tackles the problem of missing qualified talent.
3. **Bias Mitigation:** While no system can be entirely free of bias (as models are trained on human-generated data), Resume Hub shifts the focus from subjective, human-interpreted signals to an objective analysis of skills and experience content. This promotes a fairer and more standardized initial assessment.

The system provides a solid, functional foundation that proves the concept of using semantic AI for resume screening. The positive results confirm the technical and operational feasibility of the approach.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The "Resume Hub" project was undertaken to bridge the significant gaps in the traditional resume screening process. The project set out with clear objectives: to automate the screening process for efficiency, to improve accuracy through semantic understanding, and to promote fairness by reducing human bias. After a thorough process of analysis, design, implementation, and testing, it can be confidently concluded that the project has successfully achieved these objectives.

Resume Hub stands as a testament to the practical application of modern AI, specifically NLP and semantic search, in solving real-world business problems. The system effectively moves beyond the limitations of keyword matching by leveraging the power of Sentence-BERT and FAISS to understand and compare the contextual meaning of text. The developed web application is not only functional and accurate but also user-friendly, requiring minimal training for HR personnel.

This project demonstrates that it is possible to build an intelligent, accessible, and cost-effective tool that can transform the initial phase of recruitment from a tedious, error-prone task into a streamlined, data-driven process. By providing recruiters with a semantically ranked shortlist, Resume Hub empowers them to make faster, more informed, and more objective hiring decisions, ultimately helping organizations discover and hire the best talent more effectively.

6.2 Future Scope

While the current version of Resume Hub is a fully functional and effective system, there are several promising directions for future enhancement and research:

1. **Explainable AI (XAI) Integration:** A major advancement would be to integrate Explainable AI techniques. Using libraries like SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-agnostic Explanations), the system could generate detailed reports explaining *why* a candidate received a specific score. For example, it could highlight which skills were strong matches, which were missing from the job description, and how the candidate's experience level contributed to the score.
2. **Platform and Tool Integration:** To become a seamless part of the HR workflow, Resume Hub could be integrated with other platforms.
 - **LinkedIn Integration:** Allow recruiters to import candidate profiles directly from LinkedIn.

- **Calendar and Email Integration:** Automatically send interview invitations via email and schedule them directly into the recruiter's calendar (e.g., Google Calendar, Outlook).
 - **HRMS Integration:** Connect with larger Human Resource Management Systems (HRMS) like SAP SuccessFactors or Workday via APIs.
3. **Custom Model Fine-Tuning:** The accuracy of the SBERT model could be further enhanced by fine-tuning it on a domain-specific dataset. This dataset would consist of resumes and job descriptions that have been labeled by HR experts within a specific industry (e.g., IT, Healthcare, Finance). A fine-tuned model would develop a more nuanced understanding of that industry's specific jargon and skill hierarchies.
 4. **Advanced Filtering and Search:** Incorporating traditional filters (e.g., years of experience, location, education level, current company) alongside the semantic ranking would give recruiters more granular control over the final shortlist.
 5. **Bias Detection and Alerting:** Future versions could include a module that actively analyzes the job description itself for potentially biased language (e.g., gender-coded words, unnecessary age or location requirements) and alerts the user, suggesting more inclusive alternatives.
-

REFERENCES

1. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992.
 2. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535-547.
 3. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.
 4. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media, Inc.
 5. Honnibal, M., & Montani, I. (2017). spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing.
 6. Flask Documentation (2023). Pallets Projects. Retrieved from: <https://flask.palletsprojects.com/>
 7. React Documentation (2023). Meta Open Source. Retrieved from: <https://reactjs.org/>
 8. Sentence-Transformers Documentation (2023). UKPLab. Retrieved from: <https://www.sbert.net/>
 9. FAISS Documentation (2023). Facebook Research. Retrieved from: <https://faiss.ai/>
-

APPENDICES

Appendix A: User Manual

Title: A Brief Guide to Using Resume Hub

1. Accessing the System:

- Open your web browser and navigate to the Resume Hub URL (e.g., `http://localhost:3000` if running locally).
- On the login screen, enter your registered email address and password.
- Click the "SIGN IN" button. You will be redirected to the main dashboard.

2. Starting a Screening Session:

- **Step 1:** In the "Job Description" text area, paste the full text of the job description for the role you are hiring for.
- **Step 2:** In the "Candidate Resumes" section, drag and drop your resume files (PDF, DOCX, JPG, PNG) or click to browse and select them from your computer. You can select multiple files at once.
- Once both steps are complete, click the "START AI SCREENING" button.

3. Viewing Results:

- The system will process the files. Once complete, you will be automatically directed to the results page.
- Review the list of candidates, which is sorted from highest to lowest match score.
- The match score breakdown (Excellent, Good, etc.) can be used to quickly prioritize candidates for the next stage.

4. Logging Out:

- To securely end your session, simply close the browser tab or click the "Log Out" button if available.

Appendix B: Source Code and Setup

The complete source code, including the frontend React application, backend Flask server, and all configuration files, is maintained in a structured directory. The code is well-commented for ease of understanding. A `requirements.txt` file is provided for installing all necessary Python dependencies, and a `package.json` file lists all frontend dependencies.

For further academic or development purposes, the complete codebase and setup instructions can be made available upon request.

